

Sistema Automatizado de Servicios con Bash

Proyecto Integrador PPS

Gestor de usuarios, respaldos, monitoreo, servicios, red, ejecucion remota,
inventario y alertas por Telegram.

Equipo: PPS

Repositorio: Hack2-1/proyectodejavi

1. Objetivo del proyecto

El proyecto pide crear un sistema automatizado con scripts Bash para administrar servicios en GNU/Linux.

- Leer configuracion desde un archivo comun: config.txt.
- Validar entradas y manejar errores.
- Registrar acciones en logs.
- Enviar alertas y eventos por Telegram.
- Evitar ejecutar todo como root cuando no sea necesario.
- Probar los scripts y documentar su uso.

2. Solucion implementada

Se organizaron scripts independientes, pero todos comparten la misma logica:

- Se carga config.txt.
- Se validan parametros, archivos y comandos necesarios.
- Se ejecuta la tarea principal.
- Se registra el resultado en log.
- Se manda Telegram cuando existe evento importante.
- Se usa trap para cerrar limpio si se interrumpe el script.

Los archivos generados se guardan en rutas del usuario para no depender de root.

3. Arquitectura del sistema

El sistema se organizo en tres partes:

- Configuracion: config.txt y telegram.env.
- Scripts operativos: usuarios, respaldo, monitoreo, servicios, remoto, red e inventario.
- Evidencia: logs, reportes, respaldos, inventarios y mensajes de Telegram.

Flujo general:

script -> config.txt -> validacion -> accion -> log/reporte -> Telegram

4. Archivos principales

- usuarios.sh: administra usuarios.
- respaldo.sh: crea respaldos comprimidos.
- monitoreo.sh: revisa CPU, disco y memoria.
- servicios.sh: revisa y reinicia servicios caidos.
- remoto.sh: copia y ejecuta scripts en una VM por SSH.
- red.sh: revisa ping, puertos y clasifica hosts.
- inventario.sh: genera inventario de hardware y software.
- diagnostico.sh: prueba rapida para ejecucion remota.
- configurar-telegram.sh: configura token y chat id.
- test.sh: ejecuta pruebas seguras.

5. Configuración general

El archivo config.txt centraliza los parámetros:

- LOG_DIR: carpeta de logs.
- BACKUP_DIR: carpeta de respaldos.
- REPORTS_DIR: carpeta de reportes.
- INVENTORY_DIR: carpeta de inventarios.
- UMBRAL_CPU, UMBRAL_DISCO, UMBRAL_MEMORIA.
- SERVICIOS, HOSTS_PING y PUERTOS_CRITICOS.
- USUARIO_REMOTO, PUERTO_SSH y TIMEOUT_SSH.

Las credenciales reales de Telegram no se guardan en el repositorio.

6. Rutas usadas sin root

Para cumplir la buena practica de no usar root innecesariamente, el proyecto trabaja con carpetas del usuario:

Logs:

`$HOME/.local/state/sistema-servicios/logs`

Reportes:

`$HOME/.local/state/sistema-servicios/reportes`

Inventarios:

`$HOME/.local/state/sistema-servicios/inventarios`

RespalDOS:

`$HOME/respaldos/sistema-servicios`

Solo usuarios.sh y servicios.sh usan sudo cuando la accion lo necesita.

7. Configuración de Telegram

Se ejecuta:

```
bash configurar-telegram.sh
```

El script pide:

- Token del bot.
 - Chat ID del grupo.
- En la prueba se usó el grupo PPS y el chat id negativo del grupo.

Las credenciales quedan guardadas en:

```
$HOME/.config/sistema-servicios/telegram.env
```

Para probar alertas:

```
bash monitoreo.sh -c 0 -d 0 -m 0
```


8. usuarios.sh

Funcion principal: crear, eliminar, modificar y listar usuarios.

Ejecucion:

bash usuarios.sh

Muestra un menu:

- 1. crear usuario
- 2. eliminar usuario
- 3. modificar shell
- 4. listar usuarios

Valida si el usuario existe o no existe, registra en usuarios.log y manda notificacion de Telegram por cada accion realizada.

9. respaldo.sh

Funcion principal: comprimir directorios con tar y verificar el respaldo.

Ejecucion con config.txt:

```
bash respaldo.sh
```

Ejecucion con un directorio especifico:

```
bash respaldo.sh /home/michael/Documentos
```

El script:

- Valida que los directorios existan.
- Crea un .tar.gz.
- Verifica que el archivo exista y tenga tamano mayor a cero.
- Registra el resultado y manda ruta, tamano y fecha por Telegram.

10. monitoreo.sh

Funcion principal: leer CPU, disco y memoria.

Ejecucion normal:

```
bash monitoreo.sh
```

Ejecucion con umbrales personalizados:

```
bash monitoreo.sh -c 80 -d 85 -m 90
```

Forzar alerta para la demo:

```
bash monitoreo.sh -c 0 -d 0 -m 0
```

Siempre registra la lectura. Si se supera un limite, manda alerta a Telegram con saltos de linea legibles.

11. servicios.sh

Funcion principal: revisar servicios configurados o recibidos por parametro.

Ejecucion con config.txt:

```
bash servicios.sh
```

Ejecucion con servicios especificos:

```
bash servicios.sh ssh nginx
```

El script:

- Usa systemctl para revisar si esta activo.
- Si esta caido, intenta reiniciar.
- Usa sudo solo para el reinicio si hace falta.
- Registra el estado y manda resultado por Telegram.

12. remoto.sh y VM

Funcion principal: copiar y ejecutar un script en una maquina remota por SSH.

VM usada en prueba:

IP: 192.168.1.72

Usuario: michael

Ejecucion:

bash remoto.sh diagnostico.sh hosts.txt

El script copia temporalmente el archivo a:

/tmp/diagnostico.sh

Luego lo ejecuta, captura la salida, borra el temporal y genera reportes.

13. Reportes de remoto.sh

remoto.sh genera:

- Un reporte general de la ejecucion remota.
- Un reporte individual por cada host visitado.

Los reportes se guardan en:

\$HOME/.local/state/sistema-servicios/reportes

El reporte incluye:

- Fecha.
- Script local ejecutado.
- Servidor y conexion usada.
- Estado de SSH, copia y ejecucion.
- Salida capturada del script remoto.

14. red.sh

Funcion principal: revisar conectividad y puertos.

Ejecucion con config.txt:

```
bash red.sh
```

Ejecucion con archivo externo:

```
bash red.sh -f hosts.txt -p "22 80 443"
```

Clasifica cada host como:

- accesible
- parcialmente accesible
- sin respuesta

Si hay fallas, manda alerta por Telegram.

15. inventario.sh

Funcion principal: generar inventario de hardware y software.

Ejecucion:

```
bash inventario.sh
```

Incluye:

- Sistema operativo, kernel y hostname.
- Modelo de CPU y nucleos.
- RAM total, usada y disponible.
- Uso de disco por particion.
- Interfaces, usuarios, servicios y software instalado.

Tambien manda resumen por Telegram y guarda reporte en INVENTORY_DIR.

16. Cron y automatizacion

El proyecto incluye crontab.txt con ejemplos para programar tareas:

- RespalDOS diarios.
- Monitoreo cada 5 minutos.
- Revision de servicios cada 10 minutos.
- Monitoreo de red cada 15 minutos.
- Inventario diario.
- Diagnostico remoto.

Se usa con:

crontab -e

17. Comandos clave para la demo

```
bash configurar-telegram.sh
bash monitoreo.sh -c 0 -d 0 -m 0
bash respaldo.sh /tmp/carpetas_prueba
bash inventario.sh
bash red.sh -f hosts.txt -p "22 80"
bash remoto.sh diagnostico.sh hosts.txt
bash test.sh
```

Estos comandos muestran alertas, reportes, respaldos, inventario y ejecucion remota.

18. Evidencias que se deben mostrar

1. Telegram configurado en el grupo PPS.
2. Alerta de monitoreo con CPU, disco y memoria.
3. Respaldo generado con ruta, tamaño y fecha.
4. Inventario generado y resumen enviado.
5. Alerta de red con host parcial o sin respuesta.
6. Ejecución remota en VM Debian 192.168.1.72.
7. Reporte remoto legible en REPORTS_DIR.
8. test.sh con todas las pruebas pasadas.

19. Problemas resueltos

Durante la integracion se resolvieron detalles reales del proyecto:

- Telegram estaba usando un bot incorrecto; se configuro el bot PPS.
- El Chat ID del grupo era negativo; se obtuvo desde getUpdates.
- Los saltos de linea salian como texto \n; se corrigio el formato.
- SSH tenia una llave vieja de la VM; se actualizo known_hosts.
- Los reportes remotos eran dificiles de leer; se mejoro el formato.
- Se evitaron rutas /var y /etc por defecto para no depender de root.

20. Pruebas y evidencia

Se creo test.sh para validar de forma segura:

bash test.sh

Resultado final:

total: 29

pasadas: 29

falladas: 0

todas las pruebas pasaron

Tambien se verifico:

- Telegram en el grupo PPS.
- Ejecucion remota real contra Debian VM.
- Generacion de reportes, respaldos e inventarios.
- GitHub actualizado con el commit final.

21. Cierre

El sistema cumple los puntos principales del proyecto:

- Scripts funcionales e independientes.
- Configuración centralizada.
- Logs y reportes.
- Notificaciones en tiempo real por Telegram.
- Ejecución remota por SSH.
- Buenas prácticas: validaciones, trap y uso mínimo de root.

Demostración sugerida:

1. monitoreo con alerta.
2. respaldo.
3. inventario.
4. remoto contra la VM.
5. revisión de reportes generados.